

Previous Lecture Notes Commands :

```
```sh
terraform destroy --auto-approve
```
```

```
```sh
terraform destroy --auto-approve -target=aws_instance.my_instance[1]
```
```

Here are the complete notes for your new batch of images, continuing from where we left off — now covering the actual commands, syntax, state file, and workflow. I've arranged these in the order you'd naturally learn them: syntax first, then commands, then state management.

PROVIDER BLOCK & HCL SYNTAX BASICS

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

vi main.tf

```
provider "aws" {
  region = "ap-south-1"
}
```

provider is block
aws is label
{ } inside is arguments

```
resource "aws_instance" "myinstance" {
  ami = "ami-02ddb77f8f93ca4ca"
```

--- ONE LINE TO REMEMBER ---

In HCL, everything is a block with a label and arguments inside curly braces.

--- ANALOGY / STORY ---

Think of a block like a form at the doctor's office. The form type is "provider" (block), the patient name is "aws" (label), and the details you fill in like age and symptoms are the arguments inside the curly braces.

--- SIMPLE EXPLANATION ---

Terraform code is written in HCL, which uses blocks to organize everything. A block starts with a type (like provider or resource), followed by a label in quotes (like "aws"), and then arguments inside curly braces. The provider block tells Terraform which cloud to connect to and where. The resource block tells Terraform what to create, like an EC2 server.

--- HOW IT WORKS (STEP BY STEP) ---

Open a .tf file → Write provider block with cloud label and settings → Write resource blocks for what you want to build → Save and run Terraform commands

--- INTERVIEW-READY ANSWER ---

Terraform uses HashiCorp Configuration Language which is built around blocks. A provider block configures the cloud plugin with its label and arguments, while resource blocks define the actual infrastructure. For example, provider "aws" with a region argument connects Terraform to AWS in a specific region, and a resource "aws_instance" block creates a VM with specified parameters like AMI ID.

--- QUICK MEMORY HOOK ---

"Block = Container, Label = Name tag, Braces = The details inside."

--- COMMANDS / CODE (IF ANY) ---

```
``hcl
provider "aws" {                # block type = provider, label = aws
  region = "ap-south-1"         # argument inside braces
}

resource "aws_instance" "myinstance" { # block type = resource, labels = aws_instance,
myinstance
  ami = "ami-02ddb77f8f93ca4ca"      # argument
}
``
```

--- INTERVIEW TRAPS & FOLLOW-UPS ---

Q: What are the three parts of a Terraform block?

A: Block type, label(s), and arguments inside curly braces.

Q: What does the provider block do?

A: It configures the plugin that connects Terraform to a specific cloud provider like AWS.

Q: Can you have multiple provider blocks?

A: Yes, you can configure multiple providers or even multiple instances of the same provider with different aliases.

=====

=====

TERRAFORM CORE COMMANDS: FMT & INIT

=====

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

-- terraform fmt [used to format your configuration files into a canonical format and style]

terraform fmt -recursive -- for all files

-- terraform init

Whenever you have a new or existing Terraform directory (containing your Terraform configuration files), you need to run terraform init to prepare that directory for other Terraform commands.

Provider Plugins: Terraform uses plugins to interface with cloud providers (like AWS, Azure, Google Cloud, etc.). The init command checks the configuration files to see which providers you're using and fetches the required provider plugins.

Provider Versions: If you've specified a particular version of a provider in your configuration, terraform init will download that version. If not, it'll get the latest compatible version.

-- ls -al

-- cd .terraform ---> Navigate and see aws plugin with version

-- cd

--- ONE LINE TO REMEMBER ---

terraform init is the first command you run in any Terraform directory to download plugins and prepare the workspace.

--- ANALOGY / STORY ---

terraform fmt is like an auto-correct for your essay — it fixes spacing and indentation so your code looks clean. terraform init is like setting up your toolbox before starting work; it checks what tools (plugins) you need and downloads them.

--- SIMPLE EXPLANATION ---

Before Terraform can do anything, it needs to initialize the directory. terraform init reads your files, figures out which cloud providers you are using, and downloads the necessary plugins. If you specified exact versions, it downloads those; otherwise it gets the latest compatible one. terraform fmt is a formatting command that cleans up your code style automatically, and -recursive applies it to all files in subdirectories.

--- HOW IT WORKS (STEP BY STEP) ---

Write code → Run terraform fmt to clean formatting → Run terraform init to download provider plugins → Inspect .terraform directory to verify plugins

--- INTERVIEW-READY ANSWER ---

terraform init is the mandatory first step in any Terraform workflow. It initializes the working directory by downloading the provider plugins and modules specified in the configuration. If version constraints are defined, it respects them; otherwise it pulls the latest compatible version. I also use terraform fmt to enforce consistent code style across the team, often with -recursive to format entire projects.

--- QUICK MEMORY HOOK ---

"fmt = Pretty print, init = Plugin sprint."

--- COMMANDS / CODE (IF ANY) ---

```
```bash
terraform fmt # Format current file/directory
terraform fmt -recursive # Format all .tf files recursively
terraform init # Initialize directory and download plugins
ls -al # List files to see .terraform directory
cd .terraform # Navigate to see downloaded provider plugins
```
```

--- INTERVIEW TRAPS & FOLLOW-UPS ---

Q: What happens if you run terraform plan before terraform init?

A: It will fail because Terraform hasn't downloaded the provider plugins yet.

Q: Does terraform init need to be run every time?

A: No, only when you start a new project, add new providers, or change provider versions.

Q: What does terraform fmt -recursive do?

A: It formats all Terraform configuration files in the current directory and all subdirectories.

=====

=====

TERRAFORM WORKFLOW: PLAN, APPLY, DESTROY & SYMBOLS

=====

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

+ : Creating
- : Deleting
~ : Update

I - Init

P - Plan

A - Apply

D - Destroy

--- ONE LINE TO REMEMBER ---

IPAD: Init → Plan → Apply → Destroy, with + for create, - for delete, and ~ for modify.

--- ANALOGY / STORY ---

Think of Terraform like a construction project. Init is getting your permits and tools. Plan is the blueprint review where you see what will be built (+), torn down (-), or renovated (~). Apply is when the bulldozers and cranes actually start working. Destroy is demolition day.

--- SIMPLE EXPLANATION ---

Terraform shows symbols in its plan output to tell you what will happen: plus (+) means creating something new, minus (-) means deleting something old, and tilde (~) means updating something that already exists. The core workflow follows IPAD: Init to set up, Plan to preview changes, Apply to execute them, and Destroy to remove everything.

--- HOW IT WORKS (STEP BY STEP) ---

terraform init → terraform plan (see +, -, ~ symbols) → terraform apply (confirm and build) → terraform destroy (remove when needed)

--- INTERVIEW-READY ANSWER ---

The standard Terraform workflow is IPAD: Init, Plan, Apply, Destroy. During terraform plan, Terraform previews changes using symbols — plus for resources to be created, minus for resources to be destroyed, and tilde for in-place updates. I always run plan before apply to review changes, and I use destroy carefully since it removes all managed infrastructure.

--- QUICK MEMORY HOOK ---

"IPAD your infrastructure: Init, Plan, Apply, Destroy."

--- COMMANDS / CODE (IF ANY) ---

```
```bash
terraform init # I - Initialize
terraform plan # P - Preview changes (+ create, - delete, ~ update)
terraform apply # A - Execute changes
terraform destroy # D - Remove all resources
```
```

--- INTERVIEW TRAPS & FOLLOW-UPS ---

Q: What does the ~ symbol mean in terraform plan output?

A: It means Terraform will update an existing resource in place rather than recreating it.

Q: Can you skip terraform plan and go directly to apply?

A: Yes, apply runs an implicit plan first, but it's a best practice to run plan separately to review changes.

Q: Is terraform destroy reversible?

A: No, destroy permanently deletes resources. You should always review what it plans to delete before confirming.

THE .TERRAFORM DIRECTORY

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

.terraform contains lots of information (providers plugins will be stored in this directory)

-- cd .terraform

--- ONE LINE TO REMEMBER ---

The .terraform directory is Terraform's local toolbox where it stores downloaded provider plugins and modules.

--- ANALOGY / STORY ---

The .terraform folder is like the supply closet in an office. After terraform init runs, it stocks the closet with exactly the tools (plugins) your project needs. If you look inside, you'll find the AWS plugin, version info, and other dependencies.

--- SIMPLE EXPLANATION ---

When you run terraform init, Terraform creates a hidden folder called .terraform in your project directory. Inside, it stores all the provider plugins and module code it downloaded. You can navigate into this directory to verify which plugin versions are installed. This folder should not be committed to Git — it can be regenerated anytime with terraform init.

--- HOW IT WORKS (STEP BY STEP) ---

Run terraform init → Terraform creates .terraform directory → Downloads provider plugins into it → Terraform reads these plugins during plan and apply

--- INTERVIEW-READY ANSWER ---

The .terraform directory is generated when you run terraform init. It acts as a local cache for provider plugins and modules. I verify plugin versions by navigating into this directory after initialization. It's typically added to .gitignore since it can be reconstructed and may contain environment-specific paths.

--- QUICK MEMORY HOOK ---

".terraform = The plugin warehouse. Don't commit it to Git."

--- COMMANDS / CODE (IF ANY) ---

```
```bash
ls -al # View .terraform directory
cd .terraform # Enter to inspect provider plugins and versions
```
```

--- INTERVIEW TRAPS & FOLLOW-UPS ---

Q: Should you commit the .terraform directory to version control?

A: No, it contains downloaded plugins and cached files that can be regenerated with terraform init.

Q: What happens if you delete the .terraform directory?

A: You just need to run terraform init again to re-download the providers and modules.

=====

=====

TERRAFORM STATE FILE (LOCAL VS REMOTE)

=====

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

STATE FILE

=====

What is state and why is it important in Terraform?

"Terraform must store state about your managed infrastructure and configuration. This state is used by Terraform to map real world resources to your configuration, keep track of metadata, and to improve performance for large infrastructures. This state file is extremely important; it maps various resource metadata to actual resource IDs so that Terraform knows what it is managing. This file must be saved and distributed to anyone who might run Terraform."

Local State and Remote State:

"By default, Terraform stores state locally in a file named terraform.tfstate. When working with Terraform in a team, use of a local file makes Terraform usage complicated because each user must make sure they always have the latest state data before running Terraform and make sure that nobody else runs Terraform at the same time."

"With remote state, Terraform writes the state data to a remote data store, which can then be shared between all members of a team." Ex : S3

-- cat terraform.tfstate

-- terraform state list

Terraform command used to list all the resources that are currently being tracked in the Terraform

--- ONE LINE TO REMEMBER ---

The state file is Terraform's memory — it maps your code to real cloud resources so Terraform knows what it owns.

--- ANALOGY / STORY ---

Imagine Terraform is a landlord. The state file is the property register that lists every apartment (resource) Terraform owns, along with the exact address (resource ID). Without this register, Terraform wouldn't know which apartments are his if he walked down the street. Local state is keeping the register in your personal drawer; remote state is keeping it in a bank vault everyone can access.

--- SIMPLE EXPLANATION ---

Terraform keeps a state file to remember what it has created in the cloud. This file maps the resources in your code to actual resource IDs in AWS or other providers. By default, it saves this as terraform.tfstate on your local machine. But in a team, everyone needs the same state file, and two people running Terraform at once can break things. Remote state solves this by storing the file in a shared location like S3, so the whole team stays in sync.

--- HOW IT WORKS (STEP BY STEP) ---

Terraform creates resources → Saves their details to terraform.tfstate → Next time it runs, it compares code against state file → Makes only the necessary changes

--- INTERVIEW-READY ANSWER ---

Terraform uses a state file to track the mapping between configuration and real-world resources. By default, this is stored locally as terraform.tfstate, but for team collaboration I configure remote state in backend.tf — typically using S3 with state locking via DynamoDB. This ensures everyone works from the same state and prevents conflicts. I use terraform state list to inspect tracked resources.

--- QUICK MEMORY HOOK ---

"No state file = Terraform has amnesia. Remote state = Team shared memory."

--- COMMANDS / CODE (IF ANY) ---

```
```bash
cat terraform.tfstate # View the raw state file contents
terraform state list # List all resources currently tracked in state
```
```

--- INTERVIEW TRAPS & FOLLOW-UPS ---

Q: Why is the Terraform state file so important?

A: It maps resource metadata to actual cloud resource IDs, enabling Terraform to track, update, and destroy only what it manages.

Q: What are the problems with local state in a team?

A: Team members must manually share the latest state file and ensure no one runs Terraform simultaneously to avoid conflicts.

Q: What is a common remote state backend?

A: AWS S3 is commonly used, often with DynamoDB for state locking to prevent simultaneous writes.

Q: What does terraform state list show?

A: It displays all resources currently tracked in the state file, making it easy to audit managed infrastructure.

=====

That's your complete Day 1 revision set, buddy! 🎯

From "What is Terraform" all the way to "How state file works" — everything is here in the order you'll actually use it. Copy-paste this straight into your Google Docs and you're golden for the interview. Let me know when you get Day 2 notes!

Perfect! All images received and analyzed. Let me now generate your complete, organized notes. 🚀

TERRAFORM VARIABLES — COMPLETE NOTES

(Organized: Fundamentals → Types → Practical Examples → Advanced Usage)

=====

TOPIC 1: TERRAFORM VARIABLES — INTRODUCTION

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

VARIABLES

In Terraform, variables are used to make configurations more dynamic and reusable

Terraform variables are a core feature that allows you to parameterize your Terraform configurations.

Its a block in terraform used to define the variables

By using variables, you can make your code more flexible, reusable, and easier to manage.

Variables enable you to customize your Terraform deployments without hardcoding values directly into your configuration files.

Types of Terraform Variables

Input Variables: These allow you to pass values into Terraform configurations. They are defined using the variable block.

Output Variables: These are used to return values from your Terraform configurations after they have been applied, often used for
----- sharing data between different configurations or modules.

Local Variables: These are used to assign values to an expression or value within a configuration for reuse, improving readability and
----- maintainability.

--- ONE LINE TO REMEMBER --- Variables in Terraform are like function parameters — they let you write one configuration and reuse it for dev, test, and prod without touching the code.

--- ANALOGY / STORY --- Think of a restaurant menu template. The template (main.tf) stays the same — grilled item, sauce, side dish. But what goes in those slots changes every day. Variables are those changeable slots. You don't reprint the menu every day; you just swap the values.

--- SIMPLE EXPLANATION --- In Terraform, if you write `region = "ap-south-1"` directly in your file, that's called hardcoding. If tomorrow you want `us-east-1`, you have to open the file

and change it manually. Variables solve this — you define a placeholder, and you can pass any value to it without touching the main code.

There are 3 types:

- **Input Variables** → You pass values IN (like function arguments)
- **Output Variables** → Terraform shows values OUT after apply (like return values)
- **Local Variables** → Internal shortcuts within the same file (like local constants)

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → You define a variable block with a name, type, and optional default value Step 2 → In your resource block, instead of hardcoding, you write `var.variable_name` Step 3 → When you run `terraform apply`, Terraform reads the variable value (from default, tfvars file, or command line) Step 4 → Terraform substitutes the value and builds your infrastructure

--- INTERVIEW-READY ANSWER --- Terraform variables are used to make infrastructure configurations dynamic and reusable. Instead of hardcoding values like instance type or region directly in the code, you define variable blocks and reference them using `var.variable_name`. There are three types — Input variables (pass values in), Output variables (expose values after apply), and Local variables (reuse expressions within a file). This allows the same Terraform code to deploy to dev, test, or prod just by changing the variable values.

--- QUICK MEMORY HOOK --- "Variables = Don't hardcode. Use var.name and change values freely."

--- COMMANDS / CODE (IF ANY) ---

Basic variable block structure

```
variable "instance_name" {  
  description = "Name of the EC2 instance"  
  type        = string  
  default     = "TF-Server"  
}
```

How to use a variable inside a resource

```
resource "aws_instance" "myinstance" {  
  tags = {  
    Name = var.instance_name # referencing the variable  
  }  
}
```

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: What is the difference between Input and Output variables? A: Input variables pass values INTO Terraform (like arguments). Output variables expose values OUT of Terraform after apply (like return values from a module).

Q: What happens if you don't provide a default value in a variable block? A: Terraform will prompt you to enter the value interactively during `terraform apply`, unless you pass it via a `tfvars` file or command line flag.

Q: Can you use variables inside another variable? A: No. You cannot reference a variable inside another variable block. For that, you use Local values (`locals {}`).

=====

=====

TOPIC 2: TERRAFORM VARIABLE TYPES

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

Variable Types

Terraform supports several types of variables:

string: A sequence of characters ("hello", "world")

number: Any numeric value (10, 3.14, -5)

bool: A boolean value (true or false).

list(type): An ordered list of elements (["a", "b", "c"])

map(type): A key-value pair mapping ({ key1 = "value1", key2 = "value2" })

set(type): A unique unordered collection of elements (["a", "b", "c"])

object({...}): A structured object with named attributes

tuple([types]): A fixed sequence of elements with different types

--- ONE LINE TO REMEMBER --- Terraform has 8 variable types — from simple strings and numbers to complex maps, lists, sets, objects, and tuples.

--- ANALOGY / STORY --- Think of variable types like storage containers. A string is a labeled box for text. A number is a scale. A bool is a light switch (on/off). A list is a numbered shelf. A map is a dictionary. A set is a bag of unique items. An object is a form with named fields. A tuple is a fixed tray with different compartments.

--- SIMPLE EXPLANATION --- Terraform needs to know what kind of data a variable will hold. Just like in a form — some fields are for names (text), some for age (number), some for yes/no (boolean). The type tells Terraform what to expect and helps catch mistakes early.

- **string** → plain text: "hello"
- **number** → any number: 10, 3.14
- **bool** → true or false only
- **list** → ordered collection, items can repeat: ["a", "b", "a"]
- **map** → key-value pairs like a dictionary: {name = "John", age = "30"}
- **set** → like list but NO duplicates, NO guaranteed order
- **object** → a structured group of named attributes with specific types
- **tuple** → fixed-length collection where each position has a specific type

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → Declare the variable with **type = string** (or number, bool, list, etc.) Step 2 → Provide a **default** value matching that type Step 3 → Terraform validates the type when you run **terraform plan** Step 4 → If type doesn't match, Terraform throws a clear error before touching infrastructure

--- INTERVIEW-READY ANSWER --- Terraform supports primitive types — string, number, bool — and complex types like list, map, set, object, and tuple. Primitive types hold single values. Complex types hold collections. The key differences: list is ordered and allows duplicates, set is unordered with unique values only, map uses key-value pairs, object has named attributes with specific types, and tuple is a fixed sequence where each position can hold a different type.

--- QUICK MEMORY HOOK --- "String=text, Number=digits, Bool=switch, List=ordered shelf, Map=dictionary, Set=unique bag, Object=form, Tuple=fixed tray"

--- COMMANDS / CODE (IF ANY) ---

```
# string type
variable "env_name" {
  type    = string
  default = "production"
}
```

```
# number type
variable "instance_count" {
```

```

    type    = number
    default = 3
}

# bool type
variable "enable_monitoring" {
    type    = bool
    default = true
}

# list type - ordered, allows duplicates
variable "availability_zones" {
    type    = list(string)
    default = ["ap-south-1a", "ap-south-1b"]
}

# map type - key-value pairs
variable "instance_tags" {
    type = map(string)
    default = {
        Name = "WebServer"
        Env  = "Production"
    }
}

# set type - unique, unordered
variable "allowed_ports" {
    type    = set(number)
    default = [22, 80, 443]
}

# object type - structured with named attributes
variable "server_config" {
    type = object({
        name = string
        count = number
    })
    default = {
        name = "web"
        count = 2
    }
}

# tuple type - fixed positions, mixed types

```

```
variable "mixed_values" {  
  type    = tuple([string, number, bool])  
  default = ["example", 42, true]  
}
```

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: What is the difference between list and set in Terraform? A: List is ordered and allows duplicate values. Set is unordered and only stores unique values — duplicates are automatically removed.

Q: What is the difference between object and map? A: A map has all values of the same type (e.g., all strings). An object has named attributes where each attribute can have a different type.

Q: What is the difference between list and tuple? A: A list holds elements all of the same type. A tuple holds a fixed number of elements where each position can have a different type — like `[string, number, bool]`.

```
=====
```

```
=====
```

TOPIC 3: LIST TYPE — DETAILED WITH EXAMPLES

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

List Examples:

`length(list)`: Returns the number of elements in a list.

```
variable "fruits" {  
  type    = list(string)  
  default = ["apple", "banana", "cherry"]  
}
```

```
output "list_length" {  
  value = length(var.fruits)  
}
```

--- ONE LINE TO REMEMBER --- A list is an ordered, indexed collection — access items by position starting at index 0, and use `length()` to count how many items are in it.

--- ANALOGY / STORY --- A list is like a numbered waiting queue at a hospital. Patient 1 is "apple", Patient 2 is "banana", Patient 3 is "cherry". You can ask "how many patients are waiting?" — that's `length()`. You can call "Patient at position 0" — that's index access.

--- SIMPLE EXPLANATION --- A list stores multiple values in order. Think of it as a numbered array. The first item is at index 0, second at index 1, and so on. You can use built-in functions like `length()` to count items, and you access individual items using square brackets like `var.fruits[0]`.

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → Define a variable with `type = list(string)` and provide default values in [...] Step 2 → Use `var.fruits` to get the whole list Step 3 → Use `var.fruits[0]` to get the first item ("apple") Step 4 → Use `length(var.fruits)` to get the count (3 in this example)

--- INTERVIEW-READY ANSWER --- In Terraform, a list is an ordered collection of elements of the same type. You access items by their index starting from 0. The `length()` function returns the number of elements in the list. Lists are commonly used for things like availability zones, subnet IDs, or security group IDs where order matters.

--- QUICK MEMORY HOOK --- "list = ordered shelf, index starts at 0, `length()` counts the items."

--- COMMANDS / CODE (IF ANY) ---

```
# Define a list variable
variable "fruits" {
  type    = list(string)
  default = ["apple", "banana", "cherry"]
}

# Output - get total count of list items
output "list_length" {
  value = length(var.fruits) # outputs: 3
}

# Access individual items by index
output "first_fruit" {
  value = var.fruits[0]      # outputs: "apple"
}

output "second_fruit" {
  value = var.fruits[1]      # outputs: "banana"
}
```

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: What does the `length()` function do in Terraform? A: It returns the number of elements in a list, map, or string — a simple count function.

Q: How do you access a specific item from a list variable? A: Using index notation: `var.list_name[index]` — index starts at 0.

Q: Can a list hold mixed types like string and number together? A: No, a list requires all elements to be of the same type. For mixed types, use a tuple.

=====

=====

TOPIC 4: MAP TYPE — DETAILED WITH EXAMPLES

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

`map(type):` A key-value pair mapping (`{ key1 = "value1", key2 = "value2" }`)

```
default = {  
  Name = "WebServer"  
  Env  = "Production"  
}
```

```
# Output the entire map  
output "instance_tags_output" {  
  description = "Displays the instance tags as a map"  
  value      = var.instance_tags  
}
```

```
# Output a specific tag value (e.g., "Name")  
output "instance_name" {  
  description = "Displays the 'Name' tag from the instance tags"  
  value      = var.instance_tags["Name"]  
}
```

```
# Output all tag keys separately  
output "tag_keys" {  
  description = "Displays only the keys from the instance tags"  
  value      = keys(var.instance_tags)  
}
```

```
# Output all tag values separately  
output "tag_values" {
```

```

description = "Displays only the values from the instance tags"
value      = values(var.instance_tags)
}

```

--- ONE LINE TO REMEMBER --- A map is a key-value dictionary — access values by key name using `var.map_name["key"]`, and use `keys()` / `values()` to extract all keys or values.

--- ANALOGY / STORY --- A map is like a contact book. Each contact (key) has a phone number (value). You say "give me John's number" — that's `map["John"]`. You can also ask "give me all names" — that's `keys()`. Or "give me all numbers" — that's `values()`.

--- SIMPLE EXPLANATION --- A map stores data as pairs: a name (key) and its value. Like tagging an EC2 instance with `Name = "WebServer"` and `Env = "Production"`. You access values by their key. You can also use helper functions — `keys()` gives you all the key names, `values()` gives you all the values.

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → Define variable with `type = map(string)` and set default key-value pairs Step 2 → Use `var.instance_tags` to get the full map Step 3 → Use `var.instance_tags["Name"]` to get just the "Name" value Step 4 → Use `keys(var.instance_tags)` to list all keys, `values()` to list all values

--- INTERVIEW-READY ANSWER --- A map in Terraform is a collection of key-value pairs where all values must be of the same type. You access specific values using bracket notation like `var.map_name["key"]`. Built-in functions like `keys()` and `values()` help you extract just the keys or just the values from a map. Maps are commonly used for tags on AWS resources.

--- QUICK MEMORY HOOK --- "Map = Contact Book. Key is the name, Value is the number. `keys()` = all names, `values()` = all numbers."

--- COMMANDS / CODE (IF ANY) ---

```

# Define a map variable
variable "instance_tags" {
  type = map(string)
  default = {
    Name = "WebServer"
    Env  = "Production"
  }
}

```

```

# Output the entire map
output "instance_tags_output" {
  description = "Displays the instance tags as a map"
  value      = var.instance_tags
}

```

```

}

# Output a specific value by key
output "instance_name" {
  description = "Displays the 'Name' tag from the instance tags"
  value      = var.instance_tags["Name"] # outputs: "WebServer"
}

# Output all keys only
output "tag_keys" {
  description = "Displays only the keys from the instance tags"
  value      = keys(var.instance_tags) # outputs: ["Env", "Name"]
}

# Output all values only
output "tag_values" {
  description = "Displays only the values from the instance tags"
  value      = values(var.instance_tags) # outputs: ["Production", "WebServer"]
}

```

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: What is the difference between map and object in Terraform? A: In a map, all values must be the same type. In an object, each attribute can have a different type and each attribute is named explicitly.

Q: How do you access a value from a map variable? A: Using `var.map_name["key_name"]` — square bracket notation with the key as a string.

Q: What do `keys()` and `values()` functions return? A: `keys()` returns a sorted list of all keys in the map. `values()` returns a list of all values in the same order.

```

=====

=====

```

TOPIC 5: TUPLE TYPE — DETAILED WITH EXAMPLES

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

`tuple([types])`: A fixed sequence of elements with different types

tuple allows elements of different types in a fixed order.

```

variable "mixed_values" {
  type = tuple([string, number, bool])
}

```

```

    default = ["example", 42, true]
}

# Output the entire tuple
output "mixed_values_output" {
    description = "Displays the full tuple"
    value      = var.mixed_values
}

# Output individual elements from the tuple
output "first_element" {
    description = "First element (string)"
    value      = var.mixed_values[0]
}

output "second_element" {
    description = "Second element (number)"
    value      = var.mixed_values[1]
}

output "third_element" {          (from image - partially visible)
    description = "Third element (boolean)"
    value      = var.mixed_values[2]
}

```

--- ONE LINE TO REMEMBER --- A tuple is like a list but with fixed positions and each position can hold a different type — `[string, number, bool]` is a perfectly valid tuple.

--- ANALOGY / STORY --- A tuple is like a fixed-format ID card. Position 1 is always your Name (string), Position 2 is always your Age (number), Position 3 is always your Active status (bool). You can't swap them around, and you can't add a 4th slot.

--- SIMPLE EXPLANATION --- A list forces all items to be the same type. A tuple breaks that rule — each position has its own type. Position 0 is a string, Position 1 is a number, Position 2 is a bool. But the number of positions is fixed — you can't add or remove slots. Access works the same way as a list — using index numbers starting at 0.

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → Define variable with `type = tuple([string, number, bool])` — each type maps to a position Step 2 → Provide `default = ["example", 42, true]` — values must match position types exactly Step 3 → Use `var.mixed_values` to get the whole tuple Step 4 → Use `var.mixed_values[0]` for string, `[1]` for number, `[2]` for bool

--- INTERVIEW-READY ANSWER --- A tuple in Terraform is a fixed-length sequence where each element can have a different type. Unlike a list where all elements must be the same type, a tuple explicitly defines the type of each position. For example, `tuple([string, number, bool])` means position 0 is always a string, position 1 is always a number, and position 2 is always a boolean. Elements are accessed by index just like a list.

--- QUICK MEMORY HOOK --- "Tuple = Fixed ID Card. Each slot has its own type. Can't add new slots. Access by index."

--- COMMANDS / CODE (IF ANY) ---

```
# Define a tuple variable - mixed types at fixed positions
variable "mixed_values" {
  type    = tuple([string, number, bool])
  default = ["example", 42, true]
}
```

```
# Output the full tuple
output "mixed_values_output" {
  description = "Displays the full tuple"
  value       = var.mixed_values
}
```

```
# Access individual elements by index
output "first_element" {
  description = "First element (string)"
  value       = var.mixed_values[0] # outputs: "example"
}
```

```
output "second_element" {
  description = "Second element (number)"
  value       = var.mixed_values[1] # outputs: 42
}
```

```
output "third_element" {
  description = "Third element (boolean)"
  value       = var.mixed_values[2] # outputs: true
}
```

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: What is the key difference between a list and a tuple? A: A list requires all elements to be the same type and can be any length. A tuple has a fixed number of elements where each position has its own specific type.

Q: Can you add or remove elements from a tuple at runtime? A: No. A tuple is fixed in length and structure — the number and types of positions are defined at declaration and cannot change.

Q: When would you use a tuple over a list? A: When you need to group related but differently-typed values together in a fixed order — like a record of `[name, age, active_status]`.

=====

=====

TOPIC 6: PRACTICAL EXAMPLE — `main.tf` WITH VARIABLES

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

Examples

=====

Here in below code, we have 3 variables: `instance_count`, `instance_ami` and `instance_type`, these we can put any name `var.anyname`. `description = ""` and `type = number` are optional, use or don't use its same. `count`, `ami` and `instance_type` are predefined by Terraform cannot change.

```
vi main.tf
```

```
provider "aws" {  
  region = "ap-south-1"  
}
```

```
variable "instance_count" {  
  description = ""  
  type        = number  
  default     = 3  
}
```

```
resource "aws_instance" "myinstance" {  
  count      = var.instance_count  
  ami       = var.instance_ami  
  instance_type = var.instance_type  
  tags = {  
    Name = var.instance_name  
  }  
}
```

But its difficult to mange variables in single main.tf so now keep them in different configuration file called variables.tf
we no need to call variables.tf in main.tf, TF will automatically call the variable.tf file

Note: To delete multiple lines in vi, use ndd Ex:17dd : 17 lines will be deleted

vi main.tf

```
provider "aws" {  
  region = "ap-south-1"  
}  
  
resource "aws_instance" "myinstance" {  
  count      = var.instance_count  
  ami       = var.instance_ami  
  instance_type = var.instance_type  
  tags = {  
    Name = var.instance_name  
  }  
}
```

vi variables.tf

```
variable "instance_count" {  
  description = "*"   
  type        = number  
  default     = 1  
}
```

```
variable "instance_ami" {  
  description = "*"   
  ...  
}
```

```
variable "instance_name" {  
  description = "*"   
  type        = string  
  default     = "TF-Server"  
}
```

```
-- terraform fmt  
-- terraform plan
```

```
-- terraform apply --auto-approve
-- terraform state list
-- terraform destroy --auto-approve
```

--- ONE LINE TO REMEMBER --- Keep `main.tf` clean with only resources — move all variable definitions to a separate `variables.tf` file, and Terraform automatically picks it up without any import statement.

--- ANALOGY / STORY --- Think of a recipe book. Your `main.tf` is the recipe steps — "add 2 cups of flour, bake at X degrees." Your `variables.tf` is the ingredients list on a separate page. You don't staple them together — the chef knows to always check the ingredients page before cooking. Terraform works the same way.

--- SIMPLE EXPLANATION --- When you start with Terraform, you may put everything — provider, resources, AND variables — all in one `main.tf`. This gets messy fast. The clean approach is to split them: `main.tf` has the infrastructure resources (the "what to build"), and `variables.tf` has all the variable definitions (the "what are the inputs"). Terraform automatically reads ALL `.tf` files in the same folder, so no import is needed.

Key note from faculty: `count`, `ami`, and `instance_type` are predefined AWS resource arguments — you cannot rename these. But the variable names like `instance_count`, `instance_ami` are YOUR custom names — you can name them anything.

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → Create `main.tf` with provider block and resource block using `var.variable_name` Step 2 → Create `variables.tf` in the same folder with all variable definitions Step 3 → Run `terraform fmt` to auto-format → `terraform plan` to preview → `terraform apply --auto-approve` to deploy Step 4 → Terraform reads both files automatically and connects variable references

--- INTERVIEW-READY ANSWER --- In Terraform best practice, you separate your infrastructure code across multiple files. `main.tf` contains your provider and resource blocks. `variables.tf` contains your variable definitions. Terraform automatically reads all `.tf` files in the working directory, so there's no need to import or reference `variables.tf` explicitly. This keeps code clean, readable, and easy to manage across environments.

--- QUICK MEMORY HOOK --- "main.tf = what to build. variables.tf = what the inputs are. Terraform reads BOTH automatically."

--- COMMANDS / CODE (IF ANY) ---

```
# ---- main.tf ----
provider "aws" {
  region = "ap-south-1"
}
```

```

resource "aws_instance" "myinstance" {
  count      = var.instance_count # uses variable - count is predefined by AWS
  ami       = var.instance_ami   # uses variable - ami is predefined by AWS
  instance_type = var.instance_type # uses variable - instance_type is predefined by AWS
  tags = {
    Name = var.instance_name      # uses variable - Name tag value from variable
  }
}

```

---- variables.tf ----

```

variable "instance_count" {
  description = "*"
  type        = number
  default     = 1
}

```

```

variable "instance_ami" {
  description = "*"
  type        = string
  default     = "ami-0492447090ced6eb5"
}

```

```

variable "instance_type" {
  description = "*"
  type        = string
  default     = "t2.micro"
}

```

```

variable "instance_name" {
  description = "*"
  type        = string
  default     = "TF-Server"
}

```

Commands to run after creating the files

```

terraform fmt          # auto-format your .tf files
terraform plan         # preview what will be created
terraform apply --auto-approve # deploy without manual yes prompt
terraform state list    # list all resources in state
terraform destroy --auto-approve # destroy all resources without prompt

```

vi editor tip from faculty

To delete 17 lines in vi editor:

Type: 17dd (this deletes 17 lines at once)

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: Do you need to explicitly import or reference variables.tf inside main.tf? A: No. Terraform automatically reads all .tf files in the same working directory. No import or reference needed.

Q: What is the difference between `count` (the resource argument) and `instance_count` (the variable name)? A: `count` is a predefined Terraform meta-argument on the resource — you cannot rename it. `instance_count` is YOUR custom variable name that holds the value you pass to `count`. You can name your variable anything.

Q: What does `terraform fmt` do? A: It auto-formats your Terraform files to follow standard style — fixes indentation, spacing, and alignment. Safe to run anytime.

=====

=====

TOPIC 7: TERRAFORM TFVARS — VARIABLE FILES

--- FACULTY NOTES (EXACTLY AS WRITTEN) ---

Variables Files .tfvar

=====

TERRAFORM TFVARS:

This file allows you to separate variable definitions from the main configuration, making it easier to manage different environments and keep your codebase clean and organized.

we use tfvar files when we have multiple configurations like (prod, dev and test)

Each configuration we can write on variable file and attach it while running.

terraform.tfvars the default name for a .tfvars file

You can also create custom-named .tfvars files like dev.tfvars, test.tfvars, prod.tfvars

Example

No change in main.tf

```
vi main.tf
```

```
provider "aws" {  
    region = "ap-south-1"  
}  
  
resource "aws_instance" "myinstance" {  
    count      = var.instance_count  
    ami        = var.instance_ami  
    instance_type = var.instance_type  
    tags = {  
        Name = var.instance_name  
    }  
}
```

**** In variables.tf , remove all hardcoded values(default values) and keep it in separate tfvars file**

```
vi variables.tf
```

```
variable "instance_count" {  
    description = ""  
    type        = number  
}
```

```
variable "instance_ami" {  
    description = ""  
    type        = string  
}
```

```
variable "instance_type" {  
    description = ""  
    type        = string  
}
```

```
variable "instance_name" {  
    description = ""  
    type        = string  
}
```

**** Now create 3 different tfvars files like dev.tfvars, test.tfvars and prod.tfvars**

vi dev.tfvars vi test.tfvars and vi prod.tfvars with different instance_name

vi dev.tfvars

```
instance_count = 1
instance_ami   = "ami-0492447090ced6eb5"
instance_type  = "t2.micro"
instance_name  = "Dev-Server"
```

vi prod.tfvars

```
instance_count = 1
instance_ami   = "ami-0492447090ced6eb5"
instance_type  = "t2.micro"
instance_name  = "Prod-Server"
```

-- terraform apply --auto-approve -var-file="dev.tfvars"

-- terraform apply --auto-approve -var-file="test.tfvars"

[**This command now just rename Dev-Server to Test-Server as we are applying on same workspace (default), What is WorkSpace? will see **]
[terraform workspace list] [we can create a separate workspace for all diff env]

-- terraform destroy --auto-approve -var-file="test.tfvars"

-- terraform apply --auto-approve -var-file="prod.tfvars"

-- terraform destroy --auto-approve -var-file="prod.tfvars"

--- ONE LINE TO REMEMBER --- **.tfvars** files hold the actual values for your variables — one file per environment (dev/test/prod) — and you attach them at runtime using **-var-file="filename.tfvars"**.

--- ANALOGY / STORY --- Imagine you run the same food stall in three locations — Dev colony, Test nagar, and Prod city. The stall setup (main.tf) is identical. The recipe ingredients (variables.tf) list what's needed. But each location has its own ingredient supply sheet (dev.tfvars, prod.tfvars). When you set up a location, you just bring that location's supply sheet.

--- SIMPLE EXPLANATION --- `.tfvars` files are where you store the actual values for your variables — separately from where you declare them. Your `variables.tf` just says "I need a variable called `instance_name` of type string." Your `dev.tfvars` says "`instance_name` = Dev-Server." Your `prod.tfvars` says "`instance_name` = Prod-Server." Same code, different values, different environments. You tell Terraform which file to use when running the command.

The default file name is `terraform.tfvars` — Terraform loads it automatically. For custom names like `dev.tfvars`, you must pass it with `-var-file="dev.tfvars"`.

--- HOW IT WORKS (STEP BY STEP) --- Step 1 → Remove all `default` values from `variables.tf` (keep only type and description) Step 2 → Create separate `dev.tfvars`, `test.tfvars`, `prod.tfvars` files with actual values Step 3 → Run `terraform apply --auto-approve -var-file="dev.tfvars"` to deploy for dev Step 4 → Switch to prod by running the same command with `-var-file="prod.tfvars"`

--- INTERVIEW-READY ANSWER --- Terraform `.tfvars` files allow you to externalize variable values from your code, making the same infrastructure configuration reusable across multiple environments. You declare variables in `variables.tf` without defaults, then create separate `dev.tfvars`, `test.tfvars`, and `prod.tfvars` files with environment-specific values. You attach the right file at runtime using `-var-file="dev.tfvars"`. The default file `terraform.tfvars` is loaded automatically by Terraform without any flag.

--- QUICK MEMORY HOOK --- "tfvars = the values. variables.tf = the declarations. `-var-file` tells Terraform WHICH values to use."

--- COMMANDS / CODE (IF ANY) ---

---- variables.tf (no defaults - values come from tfvars) ----

```
variable "instance_count" {  
  description = ""  
  type        = number  
}
```

```
variable "instance_ami" {  
  description = ""  
  type        = string  
}
```

```
variable "instance_type" {  
  description = ""  
  type        = string  
}
```

```
variable "instance_name" {
```

```

description = "*"
type        = string
}

# ---- dev.tfvars ----
instance_count = 1
instance_ami   = "ami-0492447090ced6eb5"
instance_type  = "t2.micro"
instance_name  = "Dev-Server"

# ---- prod.tfvars ----
instance_count = 1
instance_ami   = "ami-0492447090ced6eb5"
instance_type  = "t2.micro"
instance_name  = "Prod-Server"

# Deploy for Dev environment
terraform apply --auto-approve -var-file="dev.tfvars"

# Deploy for Test environment
terraform apply --auto-approve -var-file="test.tfvars"

# Destroy Test environment
terraform destroy --auto-approve -var-file="test.tfvars"

# Deploy for Prod environment
terraform apply --auto-approve -var-file="prod.tfvars"

# Destroy Prod environment
terraform destroy --auto-approve -var-file="prod.tfvars"

# Check available workspaces
terraform workspace list

```

--- INTERVIEW TRAPS & FOLLOW-UPS --- Q: What is the difference between `terraform.tfvars` and a custom file like `dev.tfvars`? A: `terraform.tfvars` is the default name — Terraform loads it automatically without any flag. A custom file like `dev.tfvars` must be explicitly passed using `-var-file="dev.tfvars"` during the command.

Q: If you apply `dev.tfvars` and then apply `test.tfvars` on the same workspace, what happens?

A: Terraform updates the existing resources to match the new values (like renaming Dev-Server to Test-Server). It doesn't create a new separate environment — for that, you need separate Terraform workspaces.

Q: What is a Terraform Workspace (mentioned briefly in faculty notes)? A: A workspace is an isolated environment with its own state file. You can create separate workspaces for dev, test, and prod so they don't interfere with each other. Use `terraform workspace list` to see all workspaces and `terraform workspace new dev` to create one.

=====

QUICK REVISION SUMMARY TABLE

| CONCEPT | KEY POINT |
|------------------|---|
| Variables | Make configs dynamic. 3 types: Input/Output/Local |
| string | Text: "hello" |
| number | Digits: 10, 3.14 |
| bool | true or false only |
| list | Ordered, allows duplicates, index from 0 |
| map | Key-value pairs, access by key name |
| set | Unique values only, unordered |
| object | Named attributes, each can be different type |
| tuple | Fixed positions, each position different type |
| variables.tf | Where you DECLARE variables (no values) |
| terraform.tfvars | Where you SET values (auto-loaded) |
| dev.tfvars | Custom env file, use -var-file to load |
| var.name | How you reference any variable in code |
| length() | Count items in a list |
| keys() | Get all keys from a map |
| values() | Get all values from a map |

 **You're doing great!** These are one of the most important Terraform concepts for interviews. Once you understand variables + tfvars, you'll be able to explain environment management like a senior engineer. Keep going! 